

Programação em Python

Unidade 3

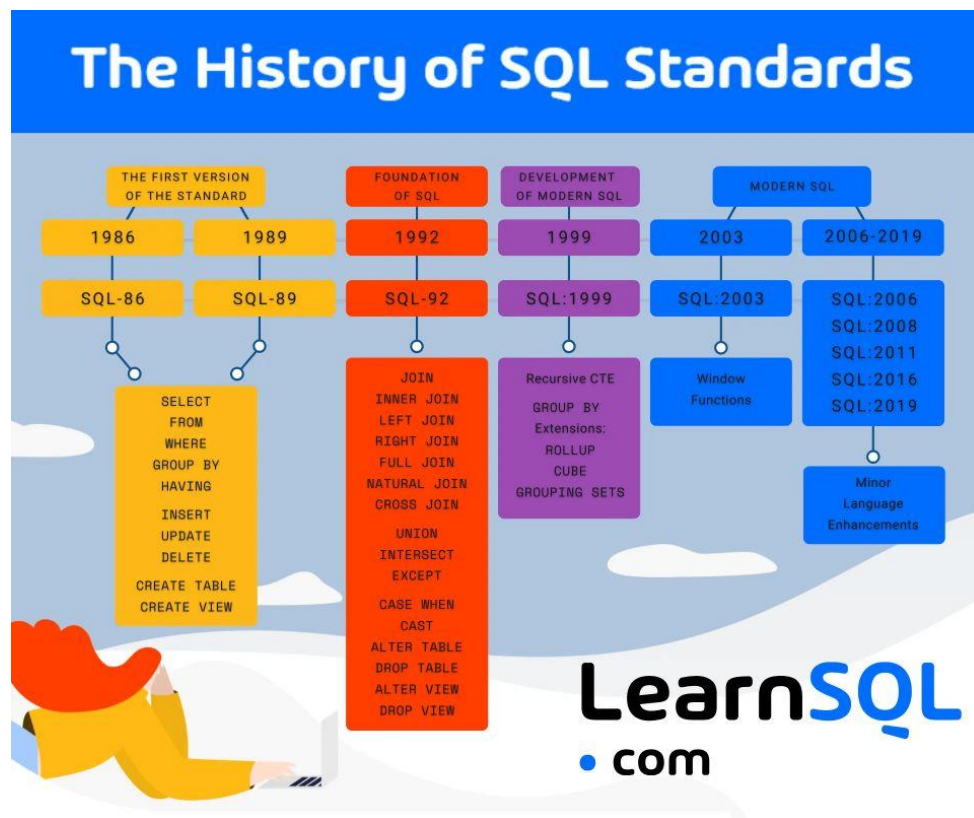
Desenvolver Aplicações Back-end para Web com Python

Sobre SQL

Python

Banco de Dados – Linguagem SQL

- A linguagem SQL está em evolução desde 1986.
- Existem diversos padrões lançados.
- Cada padrão traz um conjunto de inovações, mudanças e melhoras na linguagem.



Python

Banco de Dados – Linguagem SQL

- Pode ser dividida, basicamente, em dois conjuntos de instruções ou “**statements**”:
 - **DDL** – “**Data Definition Language**”.
 - Linguagem de definição de dados.
 - Conjunto de instruções que permitem definir o banco de dados e sua estrutura: tabelas, índices, restrições, procedimentos armazenados, etc.
 - **DML** – “**Data Manipulation Language**”.
 - Linguagem de manipulação de dados.
 - Conjunto de instruções que permitem manipular os dados armazenados no banco de dados, como, por exemplo, incluir, selecionar ou recuperar, atualizar e excluir dados nas tabelas.
 - Estes quatro processos formam a sigla “**CRUD**” – **C**reate, **R**etrieve, **U**ppdate e **D**eleate.



Saiba mais

- Sobre a linguagem SQL, visite:
<https://www.w3schools.com/sql/>
<https://www.sqltutorial.org/>
<https://www.tutorialspoint.com/sql/index.htm>

Python

Banco de Dados – Linguagem SQL

- A linguagem é composta por palavras-chave.
- Algumas das mais importantes instruções são:
 - **DDL:**
 - CREATE DATABASE – cria um banco de dados;
 - ALTER DATABASE – modifica um banco de dados;
 - CREATE TABLE – cria uma tabela, ou seja, sua estrutura no banco de dados;
 - ALTER TABLE – modifica a estrutura de uma tabela;
 - DROP TABLE – apaga uma tabela do banco de dados;
 - CREATE INDEX – cria um índice em uma tabela;
 - DROP INDEX – apaga um índice de uma tabela;
 - **DML:**
 - SELECT – recupera (seleciona) dados de uma tabela;
 - UPDATE – atualiza dados de uma tabela;
 - DELETE – apaga dados em uma tabela;
 - INSERT INTO – insere dados em uma tabela.

Pyhton

Banco de Dados – Linguagem SQL

- Alguns dos tipos de dados básicos:
 - BIT | BIT VARYING
 - BINARY(n)
 - CHAR(n) | CHARACTER(n)
 - VARCHAR(n) | CHARACTER VARYING(n)
 - INTEGER | INT | SMALLINT | BIGINT
 - DECIMAL(p,s) | DEC(p,s) | NUMERIC(p,s)
 - FLOAT | FLOAT(p) | REAL | DOUBLE PRECISION
 - DATE | TIME | TIMESTAMP
 - INTERVAL
 - BOOLEAN
 - TEXT | BLOB | CLOB | LOB | XML | JSON

Pyhton

Banco de Dados – Linguagem SQL

- NULL – valor nulo.
- Lidando com o “NULL” no MySQL:
 - `select 1 is null, 1 is not null;`
 - `select 1 = null, 1 <> null, 1 < null, 1 > null;`
 - `select 0 is null, 0 is not null, " is null, " is not null;`
- **Obs.:**
 - No MySQL, 0 ou NULL igual a “falso”.
 - Qualquer outra valor é igual a “verdadeiro”.
 - O resultado “default” de uma expressão lógica com resultado igual a 'verdadeiro' é o valor 1.

Pyhton

Banco de Dados – Linguagem SQL

- Tratamento de condições:
 - < | >
 - <= | >=
 - <> | =
 - AND | OR | NOT
 - IS [NOT] NULL
 - [NOT] LIKE
 - [NOT] IN ([,...])
 - [NOT] BETWEEN x AND y
 - [condição] ANY ([,...])
 - [condição] ALL ([,...])

Pyhton

Banco de Dados – Instruções DDL

- Criando um banco de dados:
 - `CREATE {DATABASE | SCHEMA} [IF NOT EXISTS]<nome>`
 - Ex.:
 - `CREATE DATABASE db_exemplo`
 - `CREATE SCHEMA IF NOT EXISTS schema_exemplo`
- **Obs.:**
 - Quando criamos um banco de dados devemos definir o conjunto de caracteres e outro conjunto que permite comparar os caracteres.
 - São eles:
 - “**Character Set**” – conjunto de símbolos e códigos de caracteres.
 - “**Collation**” – conjunto de regras para comparação de caracteres em um conjunto de caracteres.

Pyhton

Banco de Dados – Instruções DDL

- Apagando um banco de dados:
 - `DROP {DATABASE | SCHEMA} [IF EXISTS] <nome>`
 - Ex.:
 - `DROP DATABASE IF EXISTS db_exemplo`
 - `DROP SCHEMA schema_exemplo`
- Para acessar o banco de dados no MySQL, digite:
 - `USE <nome-bd> | <nome-schema>;`

Python

Banco de Dados – Instruções DDL

- Criando uma tabela em um banco de dados:
 - `CREATE TABLE [IF NOT EXISTS] <nome> (<definição-coluna>, ...)`
- Definição de uma coluna:
 - `<coluna> <tipo-de-dados>`
 - `[ NULL | NOT NULL ] [ UNIQUE ] [ DEFAULT valor ]`
 - `[ constraint-de-coluna | PRIMARY KEY ] [ ... ]`
 - `[, ... ]`
 - `[, PRIMARY KEY ( coluna [, ...] ) ]`
 - `[, CHECK ( condição ) ]`
 - `[, constraint-de-tabela ]`
- Ex.:
 - `create table tabela1 (nome varchar(10), endereco varchar(100))`
 - `create table tabela2 (id int not null auto_increment primary key, nome varchar(10) not null, endereco varchar(100) not null, idade int not null)`

Pyhton

Banco de Dados – Instruções DDL

- Definição de Constraints em tabelas:
 - [CONSTRAINT nome] { { UNIQUE | PRIMARY KEY } (coluna, ...) |
CHECK(condição) |
FOREIGN KEY (coluna, ...) REFERENCES tabela(coluna,...) }
- Definição de Constraints em colunas:
 - [CONSTRAINT nome] { [NOT] NULL | UNIQUE | PRIMARY KEY |
REFERENCES tabela(coluna, ...)
ON DELETE CASCADE | CHECK(condição) }

Python

Banco de Dados – Instruções DDL

- Alterando uma tabela:
 - `ALTER TABLE <nome-tabela> ([<opções>],<opções>]...)`
- Opções:
 - `ADD [COLUMN] nome-coluna definição-coluna`
 - `ADD definição-constraint-tabela`
 - `ALTER [COLUMN] nome-coluna SET`
 - `ALTER [COLUMN] nome-coluna DROP`
 - `ALTER [COLUMN] nome-coluna RESTART`
 - `DROP COLUMN nome-coluna`
 - `DROP CONSTRAINT definição-constraint`
 - `DROP FOREIGN KEY foreign-key-nome`
 - `DROP PRIMARY KEY primary-key-nome`

Pyhton

Banco de Dados – Instruções DDL

- Apagando uma tabela:
 - `DROP TABLE [IF EXISTS] <nome-tabela>`
- Criando um índice em uma tabela:
 - `CREATE [UNIQUE] [IF EXISTS] <nome-índice>`
`ON TABLE <nome-tabela> (nome-coluna [ASC | DESC], ... )`
- Apagando um índice de uma tabela:
 - `DROP INDEX [IF EXISTS] <nome-índice>`
`ON TABLE <nome-tabela>`

Pyhton

Banco de Dados – Praticando DDLs

- Crie um banco de dados, a partir do minimundo da escola “ABC do Saber”:
 - a) Crie o schema/banco de dados “[escola](#)”.
 - b) Crie e execute as instruções para a criação das tabelas e seus campos.
 - c) Crie e execute as instruções para a alteração das tabelas, de forma, que eles definam as chaves-primárias e os índices, se houver.
 - d) Ao final, junte as instruções em um grande arquivo de script SQL que realizará a criação de toda a estrutura do bando de dados e execute-o.
- Utilize o [MySQL Workbench](#) e o MySQL instalado em sua estação para o exercício.

Python

Banco de Dados – Instruções DML

- Instrução **INSERT**

- Adiciona um único registro (tupla) em uma tabela (relação).

```
INSERT INTO tabela VALUES (valor1, valor2,...)
```

```
INSERT INTO tabela (coluna1,coluna2,...) VALUES (valor1, valor2,...)
```

- Ex.:

```
INSERT INTO aluno (matricula, nome, endereco, dataNascimento)
```

```
VALUES (1000, 'Josias Jenkins', 'Rua dos Alunos, 45/302', '2000-03-15')
```

- **Obs.:**

- Especifique a tabela, colunas e a lista de valores correspondentes.
- Os valores da lista têm que estar na mesma ordem em que os campos correspondentes foram especificados na instrução.

Pyhton

Banco de Dados – Instruções DML

- Instrução **UPDATE**

- Modifica valores de atributos de um ou mais registros de uma tabela.

```
UPDATE tabela  
SET coluna = valor[, coluna = valor...]  
WHERE critério-pesquisa
```

- Ex.:

```
UPDATE aluno  
SET nome = 'Josias P. Jenkins', dataNascimento = '2003-03-14'  
WHERE matricula = 1000
```

- **Obs.:**

- Os registros podem ser selecionados através de um critério de pesquisa, caso contrário, todos os registros da tabela serão modificados
- Desta forma, procure utilizar um critério para selecionar os registros a serem modificados.

Python

Banco de Dados – Instruções DML

- Instrução **DELETE**
 - Remove um ou mais registros de uma tabela.

```
DELETE FROM tabela  
WHERE critério-pesquisa
```

- Ex.:

```
DELETE FROM aluno WHERE matricula = 1000
```

- **Obs.:**
 - Os registros podem ser selecionados através de um critério de pesquisa, caso contrário, todos os registros da tabela serão modificados.
 - Desta forma, procure utilizar um critério para selecionar os registros a serem excluídos.

Python

Banco de Dados – Instruções DML

- Instrução **SELECT**

- Recupera um ou mais registros de uma ou mais tabelas relacionadas.
- Apresenta como resultado uma ou mais colunas de um registro.

```
SELECT * | SELECT coluna1,coluna2,...  
FROM tabela  
WHERE critério-pesquisa
```

- Ex.:

```
SELECT * FROM aluno  
SELECT matricula, nome, endereco FROM aluno
```

```
SELECT nome, endereço FROM aluno  
WHERE matricula = 1000
```

- **Obs.:**

- O '*' indica que todas as colunas da(s) tabela(s) envolvida(s) serão retornados.
- Os registros podem ser selecionados através de um critério de pesquisa, caso contrário, todos os registros da tabela serão modificados.

Pyhton

Banco de Dados – Instruções DML – Instrução **SELECT**

- A palavra-chave '**AS**'
 - Permite renomear as colunas descritas na instrução **SELECT**.

```
SELECT coluna1 AS `novo nome coluna1`, coluna2 AS `novo nome coluna2`, ...  
FROM tabela WHERE critério-pesquisa
```

- Ex.:

```
SELECT matricula, nome AS 'Aluno', dataNascimento AS 'Data de Nascimento'  
FROM aluno  
WHERE matricula = 1000
```

- **Obs.:**
 - Utilize aspas simples quando o novo nome conter espaços.

Python

Banco de Dados – Instruções DML – Instrução **SELECT**

- A palavra-chave **DISTINCT**
 - Registros de tabelas podem conter valores duplicados.
 - Por exemplo, a data de cadastramento de um aluno, o mês de realização de uma prova, etc.
 - Ao recuperar valores com através da instrução **SELECT**, os valores repetidos são apresentados normalmente.
 - A palavra-chave **DISTINCT** permite apresentar o resultado de uma consulta com os valores distintos, ou seja, sem repetição dos valores no resultado da instrução.

```
SELECT DISTINCT (coluna1),coluna2,... FROM tabela
```

- Ex.:

```
SELECT DISTINCT(dataCadastramento) FROM aluno
```

Python

Banco de Dados – Instruções DML – Instrução **SELECT**

- A cláusula **ORDER BY**
 - Permite apresentar, de forma ordenada, o resultado da execução da instrução **SELECT**.

```
SELECT coluna1,coluna2,...  
FROM tabela  
ORDER BY coluna1 ASC|DESC, coluna2 ASC | DESC,...
```

- Ex.:

```
SELECT * FROM aluno  
ORDER BY dataCadastramento ASC
```

```
SELECT nome, dataNascimento FROM aluno  
WHERE dataCadastramento = '2015-05-09'  
ORDER BY nome DESC
```

- **Obs.:**
 - A ordenação acontece após a recuperação dos registros conforme a definição descrita na instrução **SELECT** executada.

Python

Banco de Dados – Instruções DML – Cláusula **WHERE**

- Operadores e Símbolos

Operador	Significado / Aplicação
= <> !=	Igual, “diferente de” ou “não igual a”.
> >=	Maior ou “maior ou igual a”.
< <=	Menor ou “menor ou igual a”.
AND	Operador lógico. Ambas as condições são verdadeiras.
OR	Operador lógico. Uma das condições é verdadeira.
NOT !	Negação de uma condição.

Operador	Significado / Aplicação
BETWEEN...AND	Entre valores. O mesmo que “>= e <=”.
IN (valor1,valor2,...)	“Em um conjunto de valores.
LIKE <i>padrão</i>	“Como” um determinado padrão.

Símbolos	Significado / Aplicação
%	Zero ou mais caracteres substituídos.
_	Apenas um caractere é substituído.

Python

Banco de Dados – Instruções DML – Instrução **SELECT**

- Exemplos de Operadores e Símbolos:

```
SELECT * FROM aluno WHERE matricula >= 100 ORDER BY dataCadastramento ASC
```

```
SELECT * FROM aluno WHERE matricula > 100 OR  
    ( dataCadastramento >= '2013-03-01' AND dataCadastramento <= '2013-03-10' )
```

```
SELECT * FROM aluno WHERE matricula >=100 AND matricula <= 200
```

```
SELECT * FROM aluno WHERE matricula BETWEEN 100 AND 200 ORDER BY nome DESC
```

```
SELECT * FROM aluno WHERE matricula NOT IN ( 212, 234, 456, 765 )
```

```
SELECT * FROM cliente WHERE primeiroNome LIKE 'Marci_'
```

```
SELECT * FROM aluno WHERE endereco LIKE '% das %'
```

Pyhton

Banco de Dados – Instruções DML – Praticando

- Abrir o [MySQL Workbench](#) e utilizar os comandos aprendidos.
 1. Utilize o banco de dados “[escola](#)” criado durante os exercícios sobre DDL.
 2. Para cada tabela, insira dados nas tabelas e crie consultas SQL para retornar os dados inseridos nas tabelas.
- **Obs.:**

Lembre-se que o banco de dados poderá ser criado a qualquer momento, a partir do script de criação desenvolvido nos exercícios sobre DDL.

Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

